

Y. Naga-tharun
192372338



SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation



B.E./ B.Tech DEGREE EXAMINATION - AUG 2026
MODEL PRACTICAL EXAMINATIONS
CSA51 - CRYPTOGRAPHY AND NETWORK SECURITY
SET 7

1. The **Hill Cipher** uses matrix multiplication for encryption. Given the plaintext "ACT" and a 3x3 Key Matrix

$$K = \begin{bmatrix} 6 & 24 & 1 \\ 13 & 16 & 10 \\ 20 & 17 & 15 \end{bmatrix}$$

- a) Convert the plaintext "ACT" into its numerical vector representation.
b) Compute the ciphertext vector using $C = (K \cdot P) \pmod{26}$. Show matrix multiplication
2. To increase security, a sender applies a **Double Transposition Cipher** to the message "MEET ME AT NOON"
- A) First, apply a simple **Rail Fence Cipher** with a depth of 2
B) Take the output of the Rail Fence and apply a **Columnar Transposition** using the key "3142"
3. Write a C program for DSA, because the value of k is generated for each signature, even if the same message is signed twice on different occasions, the signatures will differ. This is not true of RSA signatures. Write a C program for implication of this difference
4. Write a C program for SHA-3 option with a block size of 1024 bits and assume that each of the lanes in the first message block (P_0) has at least one nonzero bit. To start, all of the lanes in the internal state matrix that correspond to the capacity portion of the initial state are all zeros. Show how long it will take before all of these lanes have at least one nonzero bit.

Model practical

SET-7

NAME: Y. Naga-Charuni

REG. NO: 192372338

COURSE CODE: CSA5116

COURSE: Cryptography

1. Aim:- To implement the Hill cipher and encrypt the plaintext "ACT" using the given 3×3 key matrix.

Algorithm:-

- convert each plaintext letter into a number using $A=0, B=1, \dots, Z=25$
- Represent the plaintext as a column vector.
- multiply the key matrix K with plaintext vector P .
- Take each resulting value modulo 26.
- convert the numerical values back to letters to obtain the Ciphertext.

Pseudocode:-

START

Read plaintext P and key matrix K .

Convert P into numerical vector

$C = K \times P$

$C = C \bmod 26$

Print Ciphertext

STOP

Input:-

plaintext:- ACT

Key matrix:- $K = \begin{bmatrix} 6 & 24 & 1 \\ 13 & 16 & 10 \\ 20 & 17 & 15 \end{bmatrix}$

Output:-

Ciphertext:- pott

Result:-

Thus, the plaintext ACR was successfully encrypted using the Hill Cipher and the ciphertext obtained is pott.

2. Aim:-

To implement double transposition cipher by applying a Rail Fence cipher followed by columnar transposition.

Algorithm:-

- Read the messages and remove spaces.
- Arrange characters using Rail fence using depth 2.
- Read row-wise to get Rail fence output.
- Write output into columns using key 3142.
- Read columns in key order to obtain Ciphertext

Pseudocode:-

```
START
Read message and key
Apply Rail fence (depth=2)
Store intermediate text
Apply columnar transposition.
print final Ciphertext
STOP
```


Input:-

Message- MEET ME AT NOON

Rail Fence depth = 2

Key: 3 1 4 2

Output:-

Cipher text :- E D T A T N M N E M E O

Result:-

Thus, the message "MEET ME AT NOON" was successfully encrypted using double transposition cipher.

3. Aim:-

To implement the Digital Signature Algorithm (DSA) and demonstrate that a different random value k produces different signatures for the same message.

Algorithm:-

* select DSA domain parameters p, q and g .

* select the private key x .

* Calculate the public key $y = g^x \bmod p$.

* Calculate the hash value of the message.

* Generate a fresh random value k for every signature.

* calculate:

$$r = (g^k \bmod p) \bmod q$$

$$s = E^{-1}(H(m) + xr) \bmod q$$

* Repeat the signing process with another random k .

* Compare both signatures.

* Since k is different, the signatures will generally be different even for the same message.

Pseudocode:-

```
START
Read message M
Generate random  $k_1$  and sign M
Generate random  $k_2$  and sign M
Compare both signatures
Print signatures
STOP
```

Input:-

message = "HELLO"
private key = x
Random value k_1 for first signature.
Random value k_2 for second signature.

Output:-

message: HELLO
signature 1: (r_1, s_1)
signature 2: (r_2, s_2)
signature 1 $\neq$ signature 2

Result:-

Thus, DSA demonstrates that the same message can produce different signatures because a fresh random k is generated for every signing operations.

Aim:-

to implement the SHA-3 keccak permutation with a 1024-bit block size and determine how many rounds are required before all capacity lanes contain at least one nonzero bit.

Algorithm:-

- Initialize the keccak state matrix of 1600 bits.
- Divide the state into
 - rate = 1024 bits.
 - Capacity = 576 bits.
- Since each lane contains 64 bits:
$$576 / 64 = 9 \text{ Capacity lanes.}$$
- Initialize all capacity lanes and the rate to zero.
- Assume every lane in the first 1024-bit message block contains at least one nonzero bit.
- Apply the keccak-f permutation.
- Stop when all 9 capacity lanes contain at least one nonzero bit.

Pseudocode:-

```
START
Initialize state matrix.
Absorb message block
Apply keccak round
Check all 9 capacity lanes.
print number of rounds
STOP.
```


Input:-

Block size = 1024 bits.

Rate = 1024 bits

Capacity = 576 bits.

No. of Capacity lanes = 9

Initial capacity lanes = 0

Message block p_0 = contains nonzero

bits in its rate lanes.

Output:-

After Round 1:-

All 9 capacity lanes contain at least one nonzero bit.

No. of rounds = 1

Result:-

Thus, due to the diffusion property of the keccak-f permutation all the 9 capacity lanes become nonzero after the first round under the stated assumption.

